

Web Real-Time Communication (WebRTC)

WebRTC is an open-source framework built into every modern browser that enables **Peer-to-Peer (P2P)** communication. It allows devices to share audio, video, and data directly without routing the media through a central server.

- WebRTC
- TCP/UDP(Transmission Control Protocol)
TCP is the "Reliable Delivery" service. It is designed to ensure that every single bit of data arrives exactly as it was sent, in the correct order.

How it works: The Three-Way Handshake

Before any data is sent, TCP establishes a formal connection:

1. **SYN:** Client says, "I want to talk."
2. **SYN-ACK:** Server says, "I heard you, I'm ready."
3. **ACK:** Client says, "Great, let's start."

Key Features:

- **Error Correction:** If a packet goes missing, TCP notices and asks the sender to send it again.
- **Ordered Delivery:** If Packet #3 arrives before Packet #2, TCP holds #3 until #2 arrives so the data stays in sequence.
- **Flow Control:** It slows down the transmission if the receiver's buffer is getting full to

UDP (User Datagram Protocol)

- UDP is the "Fire and Forget" service. It is a connectionless protocol that sends data as fast as possible without checking if the receiver is actually there or if the data arrived.
- **How it works:**
 - There is no handshake. The sender just starts blasting packets (called Datagrams) at the destination IP. If a packet is lost in the "digital wind," it stays lost.
 - avoid crashing the connection.

- RTCPeerConnection

RTCPeerConnection (The "Big Boss")

This is the most important object in the WebRTC API. It handles almost everything.

- **What it does:** It manages the entire lifecycle of a connection. It handles:
 - Generating the **SDP**.
 - Connecting to **STUN/TURN** servers.
 - Encrypting the data.
 - Handling the actual transmission of the **MediaStream**.
- **Usage:** You create one `new RTCPeerConnection()` for every person you want to talk to
- RTCDataChannel

While `RTCPeerConnection` is usually for video and audio, sometimes you want to send *other* data.

- **What it does:** It creates a P2P pipe for sending text, files, or binary data.
- **Why use it?** Because it's P2P, it has much lower latency than a standard server-based chat.
- **Example:** In a WebRTC-based game, the video of the players' faces goes through the `MediaStream`, but the "Player moved to X, Y coordinates" data goes through the `RTCDataChannel` for instant response.
- **MediaStream:** Represents synchronized streams of media (audio/video). It contains `MediaStreamTrack` objects (e.g., audio track, video track) that can be linked to `<video>` or `<audio>` elements or added to `RTCPeerConnection`.
- **getUserMedia:** An API method within `navigator.mediaDevices` that prompts the user for camera/microphone access, returning a Promise that resolves to a `MediaStream`. It allows developers to set constraints like resolution or frame rate.
- **signalling:** The process of coordinating communication between peers before a direct connection is made. WebRTC does not define a specific signaling method, so developers often use WebSockets to exchange session control messages, including SDP offers/answers and ICE candidates.
- **(Session Description Protocol):** A text-based protocol that describes the multimedia session, including codecs, media types (audio/video), and network addresses. It is used in an offer-answer model: one peer sends an "offer" (SDP) outlining its capabilities, and the other responds with an "answer"
 - **Offer / Answer:** Offer: One peer (the Caller) initiates the process by creating a "description" of its media capabilities and the type of connection it wants. This description is formatted as a Session Description Protocol (SDP).
 - Answer: The receiving peer (the Callee) receives the Offer, decides which of those media capabilities it can support, and responds with its own SDP description, called the Answer.
 - **ICE (Interactive Connectivity Establishment)**

ICE is a technique used to find the available paths on the internet to connect two devices so that they can talk to each other directly.

There can be more than one path to reach the other peer and these paths are called “candidates”. ICE internally relies on STUN and TURN servers to find different candidates.

- **STUN:**

Imagine peer A wants to connect to peer B.

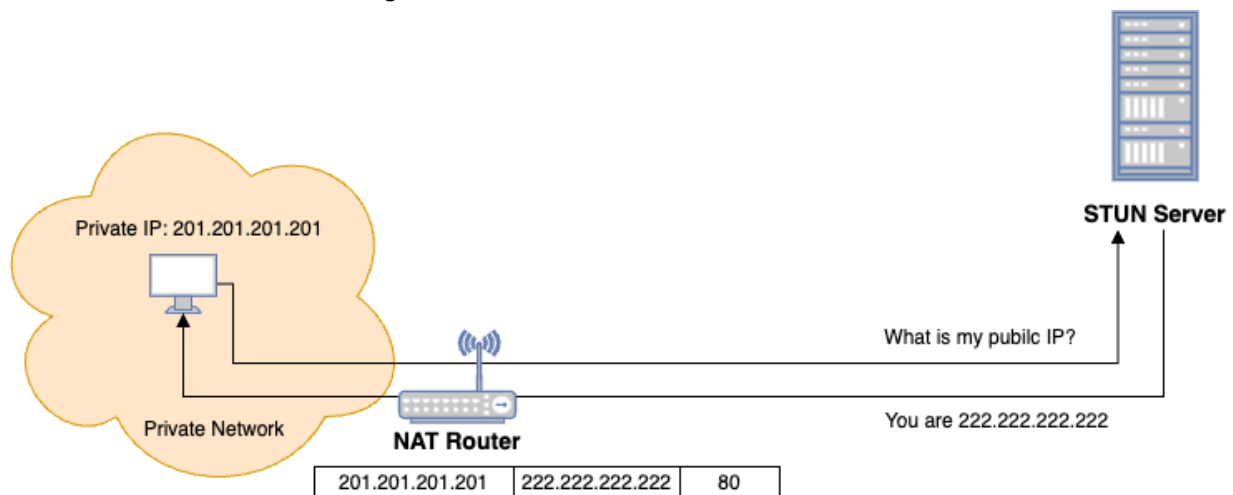
Since peer A and peer B could both be part of different private networks or behind different firewalls, they need to know their public IP addresses so that they can share the same with the other peer. This is needed for them to connect directly.

A peer does not know its public IP address since that is handled by NAT and is beyond the knowledge of the peer.

This is where the STUN server comes into the picture.

STUN servers are simply lightweight servers running on the public internet which return the IP address of the requester’s device as part of the response body.

Press enter or click to view image in full size



- Peer A wants to know its public IP address so that it can share the information with others
- Peer A connects to the STUN server and asks — “What is my public IP address”
- STUN servers can know the IP address of peer A as it will be part of the IP packet header
- STUN server sends back this public IP address as a response back to peer A

That’s it. That’s all that a STUN server does

- **TURN:**

There are times when one of the peers is behind a [symmetric NAT](#) or a firewall that has strict policies to block traffic from unknown IP addresses.

Imagine

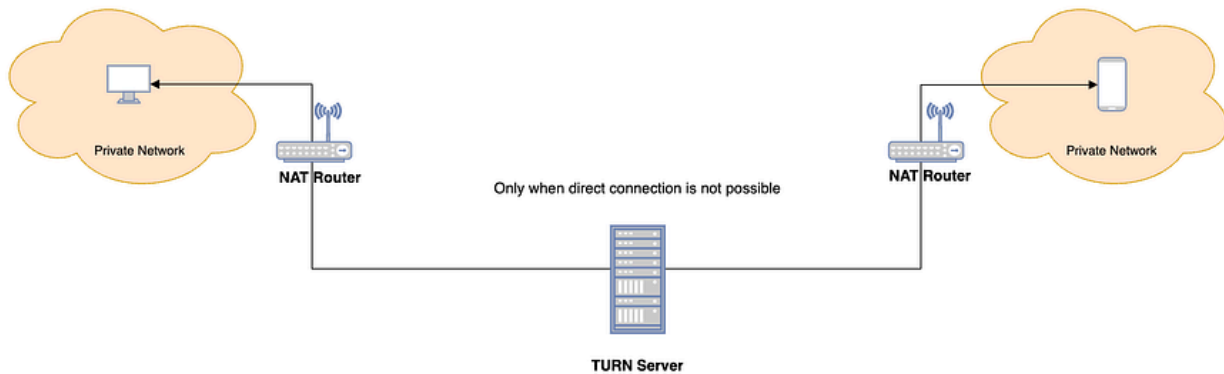
- Peer A wants to connect to peer B

- Peer A has access to peer B (thanks to signaling and registry)
- Peer A tries to connect to peer B directly
- This connection is dropped by the NAT/firewall since it detects traffic from an unknown IP source

This is where TURN servers come into the picture.

TURN servers can be thought of as a proxy. The traffic from peer A to peer B travels via the TURN server instead of going to peer B directly.

Press enter or click to view image in full size



Since TURN servers cause an additional hop to be added, they add additional latency in the data between peer A and peer B.

TURN servers are used less frequently since WebRTC will try to first establish a direct connection and then fall back to this route if that fails.

NOTE: ICE uses STUN and TURN servers to generate all the candidates

- **NAT Traversal:**

Network Address Translation, or NAT, is a transport protocol that undertakes the task of converting traffic from a device located on a private network to the internet, and vice versa. NAT traversal, also known as NAT punch-through, refers to the method of establishing direct communication between two network endpoints over a network utilizing NAT. This way, devices located behind a router can communicate directly with each other without the need for port forwarding or any other manual configurations.

- **DTLS:**

DTLS (Datagram Transport Layer Security)

Encrypts the handshake and key negotiation between peers.

- **SRTP**

SRTP (Secure Real-Time Transport Protocol)

Encrypts the actual voice and video packets once the connection is established.

- DTLS-SRTP
- Codec (VP8, VP9, H.264, AV1, Opus)

Codecs (The Translators)

A raw 1080p video stream is massive—too big for any normal internet connection. A **Codec** (Compressor-DECompressor) shrinks that data so it can be sent quickly.

- **Opus:** The **Gold Standard for Audio**. It can handle everything from low-quality voice chat to high-fidelity music. It is incredibly efficient at adjusting to changing internet speeds.
 - **H.264:** The most "compatible" video codec. Almost every phone and computer has a hardware chip dedicated to H.264, so it doesn't drain your battery.
 - **VP8 / VP9:** Open-source video codecs from Google. **VP9** is better than VP8 (higher quality at lower bitrates) and is widely used in YouTube and Google Meet.
 - **AV1:** The "Next Generation." It provides the best quality possible, but it takes a lot of CPU power to run. It is the future of WebRTC.
-
- **Bitrate:** is the amount of data processed over a specific amount of time (usually measured in **kbps** or **Mbps**).
 - **In WebRTC:** High bitrate = Sharp, beautiful video. Low bitrate = Pixelated, blurry video.
 - **Usage:** WebRTC uses **Dynamic Bitrate**. If it notices your Wi-Fi signal dropping, it will automatically lower the bitrate so the call doesn't drop, even if it makes the image look "blocky."
-
- **Jitter:** **Jitter** is the variation in the delay of received packets.
 - **Latency:**

is the time it takes for a packet to travel from your microphone to the other person's speakers.

- **The Goal:** In WebRTC, we aim for "Sub-500ms" latency. If latency gets above 500ms, people start accidentally talking over each other because the delay is too long.
 - **Usage:** This is why WebRTC prefers **UDP**. It doesn't waste time re-sending old data, which keeps the delay as low as possible.
-
- **Packet Loss:** On the internet, sometimes data just disappears (due to a bad router or a weak signal).
 - **Congestion Control:**
is the mechanism that prevents a WebRTC application from flooding the network with more data than it can handle.
How it works: If you try to send 5 Mbps of video over a 2 Mbps connection, the router will drop packets, causing the video to freeze. Congestion control detects this "traffic jam" and tells the encoder to slow down.

Usage: It ensures that your video call doesn't crash your entire home internet connection.

- **Bandwidth Estimation (BWE)**

This is the process of constantly "guessing" how much data the current connection can support.

- **How it works:** WebRTC sends "probe" packets to see how fast they arrive. If they arrive quickly, it increases the video quality. If they start arriving late or out of order, it assumes the bandwidth has decreased and lowers the quality.
- **Example:** This is why your video starts out blurry for the first 2 seconds of a call and then snaps into high definition—the **BWE** is "feeling out" the connection.

- **Mesh Network:**

This is the simplest one to set up and involves peer to peer. Basically each participant sends their stream directly to every other participant without having to go through a media server. The participant also receives streams from every other user. So if there are 4 participants. You would send your stream to the other 3 users and also receive all of their streams. As you can imagine this approach does not scale well past more than 6 users because it requires your computer to receive and process a bunch of audio and video streams. Mesh supports up to 4–6 participants and does not require a central media server which it makes it easier to set up, but does not scale.

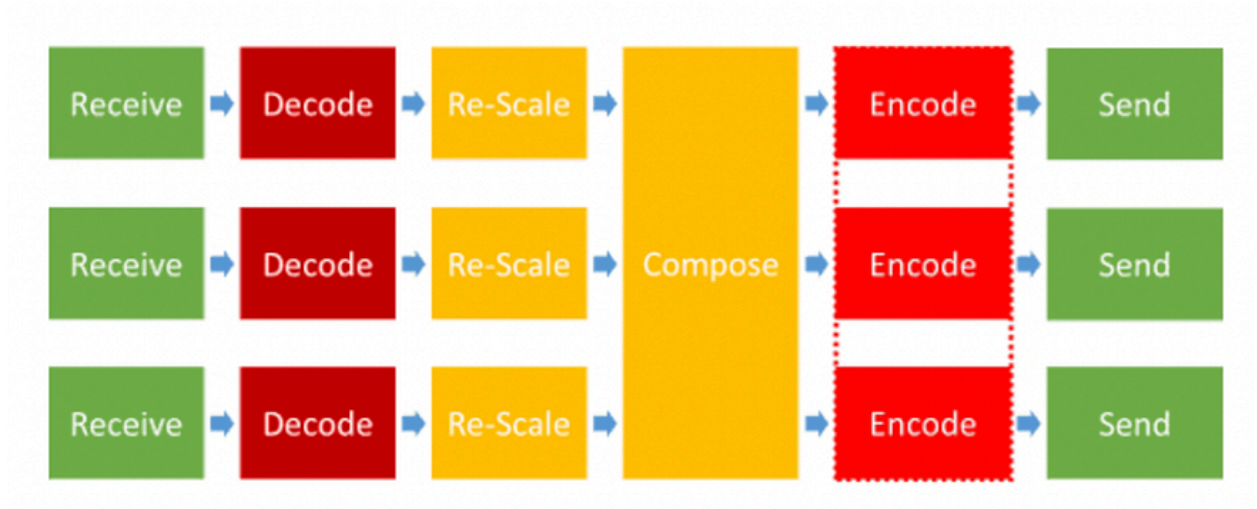
- **SFU (Selective Forwarding Unit):**

This is a more popular approach than MCU when there is a need to scale to more than 4–6 participants. This approach also involves a server. The difference is that each participant sends their stream to the server and the server sends each stream that it receives back to all of the other participants. For example if there are 6 participants, each participants will receive 5 streams from the other users. The great thing about the SFU is that it can scale and you have more flexibility over the UI. On the client you can reorganize the order of participants whereas this is not possible in the MCU. This can allow you to have custom layouts and change them while live.

- **MCU (Multipoint Control Unit):**

The MCU supports a larger number of participants than MESH. Each participants sends their stream to a server and the server decodes each participant's stream and creates a single new stream from each of the streams it has received. It finally sends this stream back to all of the participants.

Press enter or click to view image in full size



- **Trickle ICE:**

The Concept: Traditional ICE (Vanilla ICE) waits to find every possible way to connect (Local IP, Public IP, Relay IP) before sending the list to the other person. This takes a long time.

Trickle ICE sends each "candidate" the moment it is found.

- **How it works:** Instead of one big "Offer," you send a small message every time your browser discovers a new potential path.
- **The Benefit:** The connection can often be established before the browser has even finished looking for other paths. It can shave **2–5 seconds** off the call setup time.
- **WebRTC Flow:** Happens immediately after the Offer/Answer exchange.

- **Renegotiation:**

The Concept: Once a P2P call is active, the "contract" (SDP) is set. But what if the situation changes? Renegotiation is the process of updating that contract without dropping the call.

- **When it's used:**
 - * You are on an audio call and suddenly click "Start Video."
 - You switch from your front camera to your back camera.
 - A third person joins the call in a mesh network.
- **The Flow:** It triggers a new Offer/Answer cycle over the existing connection to agree on the new media parameters.

- **WebSocket (for signaling)**

The Concept: WebRTC provides the "pipe" for the video, but it doesn't provide the "phone number" to call. You need a separate server to help peers find each other. WebSockets are the standard choice for this "Signaling" server.

- **Why WebSockets?** Unlike standard HTTP (where the client must ask for data), WebSockets allow the server to "push" data to the client instantly.

- Usage in WebRTC: 1. Person A sends their SDP to the WebSocket server. 2. The server immediately "pushes" that SDP to Person B.
- Important Note: Once the P2P connection is successful, the WebSocket server can sit idle; the video data does not go through the WebSocket.

- **Peer-to-Peer (P2P):**

- The Concept: This is the core philosophy of WebRTC. It means the media (video/audio) travels directly from User A to User B.
- The Benefit: * Low Latency: No middleman server to slow things down.
 - Privacy: Since the server never "sees" the video, the conversation is more private.
 - Cost: The developer doesn't have to pay for the massive bandwidth required to stream video; the users provide the bandwidth.

- **QoS (Quality of Service):**

The Concept: QoS is a set of techniques used to ensure that high-priority data (like your boss's voice during a meeting) gets through even if the network is crowded with lower-priority data (like a background file download).

- How WebRTC uses it: WebRTC packets are "tagged" (using something called DSCP). This tells your router: "Hey, this is a live video packet. If the network gets busy, drop the email update packets first and let this one through."
- The Goal: To minimize Jitter and Latency during periods of high network congestion

WebSocket:

WebSocket is a communication protocol that enables persistent, full-duplex (two-way) interactive communication between a client (like a browser) and a server over a single TCP connection. It allows servers to push data to clients instantly, making it ideal for real-time applications like chat, live sports feeds, and stock tickers, bypassing the need for repeated HTTP requests.

Client1 < —websocket connection-----> server < —websocket connection-----> client2

1. WebSocket vs. WebRTC: The Latency Factor

While WebSockets provide "real-time" communication, they are still **Server-Client** based.

- **WebSocket Workflow:** Person A $\rightarrow$ Server $\rightarrow$ Person B.

- *Problem:* The server acts as a middleman. This adds "hops," which increases **latency** (delay), especially for heavy data like high-definition video.
- **WebRTC Workflow:** Person A $\rightarrow$ Person B.
 - *Solution:* Once the connection is established, data travels the shortest path possible between the two devices.

WebRTC(open source):

No server ,data go direct on client device.

WebRTC is P2P, but peers cannot "find" each other automatically. They need to exchange metadata first. This process is called **Signaling**.

But before the establishment of connection between clients,these client need to transfer small data between them it is very necessary. That data is called SDP (Session Description Protocol) Like a key and lock between them. Now how will they share it between them.so they share it using websocket for example when we call some one it status is like

```

A-----sdpA----->server(websocket)-----sdpA----->B
<-----sdpB-----<-----sdpB-----B

```

After handshake:

```

Person A<----->Person B

```

-calling (SDP is shared with peer)

-ringing

FastRTC (High-Level Framework)

FastRTC is designed to make real-time audio and video AI applications easy to build entirely in Python.

Benefits:

- Rapid Development: Built-in UI and easy mounting on FastAPI apps.
- AI Integration: Optimized for Text-to-Speech (TTS) and Speech-to-Text (STT) models.

Limitations:

- Abstraction: Less flexibility to customize low-level peer-to-peer behavior.
- Focus: Highly tailored for media+AI streaming, not general-purpose data P2P.

WebRTC in FastAPI (Low-Level Customization)

Benefits:

- Total Control: Granular control over the WebRTC peer connection.
- Flexibility: Adaptable to any custom signaling protocol.

Limitations:

- High Complexity: Difficult to set up NAT traversal and signaling.
- Development Speed: Slow, requires extensive knowledge of WebRTC standard

Implementation of WebRTC:

SDP

Problem :

NATS (network address transmission): change IP never remain static

p2P become complicated

ICE(interactive connectivity establishment) standard: which help client coordinate the discovery of their public facing ip address

Now Both peer generate a list of ice candidates which contain an ip address and port that peer1 can use to connect to peer2